

Vježba - Theme Toggler pomoću Contexta

U ovoj vježbi izradit ćemo malu React aplikaciju koja koristi **Context** za dijeljenje trenutne teme kroz više komponenti.

Aplikacija će imati svijetlu i tamnu temu. Korisnik će moći kliknuti gumb i prebacivati se između **light** i **dark** načina rada. Različite komponente u aplikaciji čitat će trenutnu temu iz Contexta, bez proslijeđivanja propsa kroz više razina komponenti.

Cilj vježbe

Cilj ove vježbe je razumjeti kako se **useContext** koristi kada više komponenti treba pristup istim podacima.

Učenici će vježbati:

- kreiranje Contexta pomoću **createContext**
 - izradu Provider komponente
 - spremanje theme vrijednosti u state
 - proslijeđivanje podataka kroz Context Provider
 - čitanje Context vrijednosti pomoću **useContext**
 - proslijeđivanje i vrijednosti i funkcija kroz Context
 - izbjegavanje prop drillinga
 - uvjetno stiliziranje na temelju trenutne teme
-

Scenarij

Zamislite da izrađujete malu aplikaciju koja podržava dvije vizualne teme:

light
dark

Kada je tema **light**, sadržaj treba imati svijetlu pozadinu i tamni tekst.

Kada je tema **dark**, sadržaj treba imati tamnu pozadinu i svijetli tekst.

Važno je da se tema ne proslijeđuje ručno kao prop iz **App** komponente u svaku drugu komponentu. Umjesto toga, tema se treba dijeliti pomoću Contexta.

Naziv projekta

08_React/08_Practice_ThemeTogglerContext/

Zadatak

1. Kreirajte novi React projekt

U terminalu pokrenite:

```
npm create vite@latest theme-toggler-context -- --template react
```

Zatim uđite u projekt:

```
cd theme-toggler-context
```

Instalirajte pakete:

```
npm install
```

Pokrenite aplikaciju:

```
npm run dev
```

2. Očistite početni projekt

U datoteci **src/App.jsx** obrišite početni sadržaj i pripremite osnovnu strukturu:

```
import './App.css';
```

```
function App() {  
  return (  
    <main className="app">  
      <h1>Theme Demo</h1>  
    </main>  
  );  
}
```

```
export default App;
```

U datoteci `src/App.css` obrišite postojeći CSS.

3. Kreirajte Context

Na vrhu `App.jsx` datoteke importajte potrebne funkcije iz Reacta:

```
import { createContext, useContext, useState } from "react";
import "./App.css";
```

Zatim iznad `App` komponente kreirajte `ThemeContext`:

```
const ThemeContext = createContext({
  theme: "light",
  toggleTheme: () => {}
});
```

Ovdje koristimo objekt kao početnu vrijednost Contexta.

Objekt ima dva svojstva:

```
{
  theme: "light",
  toggleTheme: () => {}
}
```

`theme` predstavlja trenutnu temu.

`toggleTheme` predstavlja funkciju za promjenu teme.

Ova početna vrijednost korisna je zato što oblikom odgovara vrijednosti koju će kasnije slati Provider. To znači da komponente koje koriste Context očekuju isti oblik podataka.

4. Napravite `ThemeProvider` komponentu

Ispod `ThemeContext` definicije napravite komponentu `ThemeProvider`.

```
function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  function toggleTheme() {
    setTheme(prev => (prev === "light" ? "dark" : "light"));
  }
}
```

```
return (  
  <ThemeContext.Provider value={{ theme, toggleTheme }}>  
    {children}  
  </ThemeContext.Provider>  
);  
}
```

Ova komponenta ima vrlo važnu ulogu.

Ona čuva state:

```
const [theme, setTheme] = useState("light");
```

Zatim definira funkciju za promjenu teme:

```
function toggleTheme() {  
  setTheme(prev => (prev === "light" ? "dark" : "light"));  
}
```

Ako je trenutna tema **"light"**, funkcija je mijenja u **"dark"**.

Ako je trenutna tema **"dark"**, funkcija je mijenja u **"light"**.

Na kraju, **ThemeProvider** vraća:

```
<ThemeContext.Provider value={{ theme, toggleTheme }}>  
  {children}  
</ThemeContext.Provider>
```

To znači da sve komponente koje se nalaze unutar **ThemeProvider** komponente mogu pristupiti vrijednostima **theme** i **toggleTheme**.

5. Napravite komponentu **ThemeToggleButton**

Sada napravite komponentu koja će čitati Context i prikazati gumb za promjenu teme.

```
function ThemeToggleButton() {  
  const { theme, toggleTheme } = useContext(ThemeContext);  
  
  return (  
    <button onClick={toggleTheme} className="theme-button">  
      Switch to {theme === "light" ? "dark" : "light"} mode  
    </button>  
  );  
}
```

Ovdje koristimo:

```
const { theme, toggleTheme } = useContext(ThemeContext);
```

To znači da komponenta uzima `theme` i `toggleTheme` direktno iz Contexta.

Primijetite da `ThemeToggleButton` ne prima props.

Ne pišemo:

```
function ThemeToggleButton({ theme, toggleTheme }) {
```

nego vrijednosti čitamo iz Contexta.

To je glavna ideja ove vježbe.

6. Napravite komponentu `ContentArea`

Sada napravite komponentu koja također čita trenutnu temu iz Contexta.

```
function ContentArea() {  
  const { theme } = useContext(ThemeContext);  
  
  const contentClass =  
    theme === "dark" ? "content-area dark" : "content-area light";  
  
  return (  
    <section className={contentClass}>  
      <h2>Content Area</h2>  
      <p>  
        This area responds to the current theme. The theme is: {theme}.  
      </p>  
    </section>  
  );  
}
```

Ova komponenta koristi samo `theme`, ne treba joj `toggleTheme`.

Na temelju trenutne teme određujemo CSS klasu:

```
const contentClass =  
  theme === "dark" ? "content-area dark" : "content-area light";
```

Ako je tema `"dark"`, element dobiva klase:

`content-area dark`

Ako je tema "light", element dobiva klase:

content-area light

7. Omotajte aplikaciju u ThemeProvider

Sada uredite App komponentu.

```
function App() {  
  return (  
    <ThemeProvider>  
      <main className="app">  
        <h1>Theme Demo</h1>  
        <ThemeToggleButton />  
        <ContentArea />  
      </main>  
    </ThemeProvider>  
  );  
}
```

Ovdje je najvažniji dio:

```
<ThemeProvider>  
...  
</ThemeProvider>
```

Sve što je unutar ThemeProvider komponente može koristiti ThemeContext.

U našem slučaju, i ThemeToggleButton i ContentArea mogu čitati temu iz Contexta.

Očekivano ponašanje aplikacije

Kada se aplikacija prvi put otvori, tema je light.

Content area treba imati svijetlu pozadinu i tamni tekst.

Gumb treba prikazivati:

Switch to dark mode

Kada korisnik klikne gumb, tema se mijenja u dark.

Content area treba promijeniti stil.

Gumb sada treba prikazivati:

Switch to light mode

Kada korisnik ponovno klikne gumb, tema se vraća na **light**.

Puno rješenje

src/App.jsx

```
import { createContext, useContext, useState } from "react";
import "./App.css";

const ThemeContext = createContext({
  theme: "light",
  toggleTheme: () => {}
});

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  function toggleTheme() {
    setTheme(prev => (prev === "light" ? "dark" : "light"));
  }

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

function ThemeToggleButton() {
  const { theme, toggleTheme } = useContext(ThemeContext);

  return (
    <button onClick={toggleTheme} className="theme-button">
      Switch to {theme === "light" ? "dark" : "light"} mode
    </button>
  );
}

function ContentArea() {
  const { theme } = useContext(ThemeContext);
```

```

const contentClass =
  theme === "dark" ? "content-area dark" : "content-area light";

return (
  <section className={contentClass}>
    <h2>Content Area</h2>
    <p>
      This area responds to the current theme. The theme is: {theme}.
    </p>
  </section>
);
}

function App() {
  return (
    <ThemeProvider>
      <main className="app">
        <h1>Theme Demo</h1>
        <ThemeToggleButton />
        <ContentArea />
      </main>
    </ThemeProvider>
  );
}

export default App;

```

src/App.css

```

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #eef2f7;
}

.app {
  max-width: 520px;
  margin: 50px auto;
  padding: 32px;
  background-color: white;
  border: 1px solid #d8dee9;
  border-radius: 16px;
}

h1 {

```

```
margin-top: 0;
text-align: center;
}

.theme-button {
display: block;
margin: 0 auto 20px;
padding: 10px 16px;
border: 1px solid #ccd3df;
border-radius: 8px;
background-color: #f7f9fc;
cursor: pointer;
font-size: 15px;
}

.theme-button:hover {
background-color: #e9eef6;
}

.content-area {
padding: 24px;
border-radius: 12px;
min-height: 180px;
border: 1px solid #d8dee9;
}

.content-area.light {
background-color: #ffffff;
color: #333333;
}

.content-area.dark {
background-color: #1a1a2e;
color: #e0e0e0;
}
```

Objašnjenje za instruktora

Ova vježba je odličan prvi susret s `useContext` jer problem nije prevelik, ali jasno pokazuje zašto Context postoji.

Bez Contexta bismo temu morali prosljeđivati kroz props:

```
<ThemeToggleButton theme={theme} toggleTheme={toggleTheme} />
<ContentArea theme={theme} />
```

U maloj aplikaciji to nije problem. Ali u većoj aplikaciji komponente mogu biti duboko ugniježdene.

Na primjer:

```
App
  Layout
    Sidebar
      Menu
        ThemeToggleButton
```

Ako `ThemeToggleButton` treba temu, morali bismo prop slati kroz `Layout`, `Sidebar` i `Menu`, čak i ako te komponente ne koriste temu. To se zove **prop drilling**.

Context rješava taj problem tako da vrijednost stavimo u Provider:

```
<ThemeContext.Provider value={{ theme, toggleTheme }}>
```

a onda je čitamo u komponenti kojoj stvarno treba:

```
const { theme, toggleTheme } = useContext(ThemeContext);
```

To znači da `ThemeToggleButton` i `ContentArea` ne ovise o tome tko im je roditelj. Dovoljno je da se nalaze negdje unutar `ThemeProvider` komponente.

Dodatni izazovi

Izazov 1: Promijenite cijelu aplikaciju, ne samo content area

Trenutno se mijenja samo `ContentArea`.

Proširite aplikaciju tako da se i glavna `.app` kartica mijenja ovisno o temi.

Možete napraviti novu komponentu `ThemedLayout` koja čita `theme` iz Contexta i postavlja klasu.

Primjer:

```
function ThemedLayout({ children }) {
  const { theme } = useContext(ThemeContext);

  const layoutClass =
    theme === "dark" ? "app app-dark" : "app app-light";
```

```
return <main className={layoutClass}>{children}</main>;
}
```

Zatim u **App** komponenti koristite:

```
function App() {
  return (
    <ThemeProvider>
      <ThemedLayout>
        <h1>Theme Demo</h1>
        <ThemeToggleButton />
        <ContentArea />
      </ThemedLayout>
    </ThemeProvider>
  );
}
```

Izazov 2: Dodajte status tekst

Dodajte novu komponentu **ThemeStatus** koja prikazuje:

Current theme: light

ili:

Current theme: dark

Komponenta mora koristiti **useContext**.

Primjer:

```
function ThemeStatus() {
  const { theme } = useContext(ThemeContext);

  return <p className="theme-status">Current theme: {theme}</p>;
}
```

Izazov 3: Dodajte još jednu komponentu koja koristi temu

Dodajte komponentu **InfoCard**.

Ona treba čitati **theme** iz Contexta i promijeniti stil ovisno o temi.

Primjer sadržaja:

Context lets deeply nested components read shared data without prop drilling.

Ovo pokazuje da više različitih komponenti može koristiti istu Context vrijednost bez propsa.

Izazov 4: Premjestite Context u zasebnu datoteku

Kada aplikacija naraste, nije idealno držati sve u `App.jsx`.

Napravite novu datoteku:

`src/ThemeContext.jsx`

U nju premjestite:

`ThemeContext`

`ThemeProvider`

Zatim ih exportajte:

```
export { ThemeContext, ThemeProvider };
```

U `App.jsx` ih importajte:

```
import { ThemeProvider, ThemeContext } from "../ThemeContext";
```